Lab U5 - Gazebo, odometry, SLAM and Nav2 in the Essentials Docker image

Aim. Run the exercises of *Robotics & ROS 2 Essentials* in **its own Docker image** (Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress, the **Andino** robot, Nav2, slam_toolbox) - something Windows cannot do natively - and check four things stage by stage: the simulated robot obeys `/cmd_vel`, your wheel odometry is right, `slam_toolbox` builds a map, and a planner plugin works inside Nav2.

Time. 30 min (+ tens of minutes the first time, for the 6.5 GB image). **Needs.** Only this folder, plus **Docker Desktop** (WSL 2 backend). The repository is downloaded by `pixi run build-essentials-docker`. Docker Desktop must be running (about 12 GB RAM for its VM, 10 GB free disk).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (`winget install --id prefix-dev.pixi -e --source winget`), and Docker Desktop with the WSL 2 backend (about 12 GB RAM for its VM, 10 GB disk); internet access for the first build.

In this folder	
`scripts\docker\`	the image build and the four test stages; `upstream\` the repository (downloaded by the build task)
`scripts\`	helper scripts and the automated test
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` provides `git` (the only tool the scripts need). Open PowerShell in this folder and run:

```
pixi install
```

Run one lab at a time, and everything from this folder.

Theory

- A Docker **image** is a frozen Linux installation; a **container** is one running copy. Everything the exercises need is inside the image, so nothing is installed on Windows.
- Gazebo normally opens a window. Here it runs **headless** on a virtual screen (Xvfb, added by a 20 MB layer on top of the repository's image). Nothing is drawn on your desktop.
- Software rendering makes Gazebo run at about **0.5 x real time**. Every test therefore measures time with the simulated clock `/clock`, never with a wall clock.
- Andino: wheel radius 0.033 m, separation 0.137 m, a 720-ray lidar. Wheel odometry is the usual differential-drive maths (`ds = (dL + dR)/2`, `dtheta = (dR - dL)/L`) with these two numbers.

Stage	Exercise	What it proves
`sim`	0-1	Gazebo loads, `/scan` `/odom` `/joint_states` `/clock` publish, `/cmd_vel 0.2` for 4 simulated s moves the robot 0.80 m

Stage	Exercise	What it proves
`odometry`	4	odometry from the wheels matches the commanded motion; Gazebo's *physical* pose differs, because simulated wheels slip
`slam`	2	`slam_toolbox` builds a map while the robot turns and drives; `map_saver_cli` writes a PNG + YAML
`nav`	5	the repository's C++ plugin runs inside Andino's Nav2 and gets its path from a Python `create_plan` service

Procedure

1. Check Docker.

```
docker --version
docker info --format "Server: {{.ServerVersion}}"
```

2. Build the images (the repository's own `docker/Dockerfile` plus the thin headless layer; seconds when cached, tens of minutes on the first run).

```
pixi run build-essentials-docker
```

3. Look inside the image. A throw-away container lists the ROS packages that belong to the robot.

```
docker run --rm robotics_essentials_ros2:headless ros2 pkg list | Select-String
"andino|slam_toolbox"
```

4. Run the stages one at a time (each starts a container, runs its checks, prints `PASS` / `FAIL` lines and removes the container).

```
powershell -NoProfile -File scripts\docker\run_essentials_docker.ps1 -Stage sim
```

```
powershell -NoProfile -File scripts\docker\run_essentials_docker.ps1 -Stage odometry
```

```
powershell -NoProfile -File scripts\docker\run_essentials_docker.ps1 -Stage slam
```

```
powershell -NoProfile -File scripts\docker\run_essentials_docker.ps1 -Stage nav
```

5. Look at the map that SLAM built (opens in your picture viewer; white = free, black = walls, grey = unknown).

```
Invoke-Item build\docker_out\andino_map.png
```

Observations (recorded; the real-time factor depends on your CPU)

Stage	Recorded result
`sim`	6 checks; real-time factor 0.65; `/scan` 720 rays; **0.80 m** in 4 simulated seconds
`odometry`	6 checks; our odometry off by 0.003 m / 0.000 rad from the commanded motion; Gazebo's physical pose moved 0.87 m and differs from the ideal (slip)

Stage	Recorded result
`slam`	7 checks; map about 395 x 301 cells at 0.05 m (44 000 free, 560-600 occupied); `andino_map.png` (13.7 kB) + `.yaml` written to `build\docker_out`
`nav`	13 checks; plugin `custom_nav2_planner/CustomPlanner` loaded, goal 1.6 m ahead reached (0.23 m off), `create_plan` called 14 times, plan = straight line (0.000 m deviation)

Verification (all four stages in one command, 7-8 minutes)

```
pixi run test-essentials-docker
```

Recorded: 32 of 32 checks.

Questions

1. Why is Gazebo's own `/odom` farther from the commanded motion than your wheel odometry? Which of the two says where the robot *really* is?
2. Why do the tests use `/clock` instead of `Start-Sleep`?
3. What would be different if the container had no virtual display? (Gazebo's GUI could not start.)
4. Which Docker option would you add to see Gazebo's window on Linux (hint: `DISPLAY`, X11 socket)? Why is that not possible directly on Windows?
5. *(Extension)* Open `scripts\docker\run_stage.sh`: which command starts Gazebo, and which lines wait for the robot to be ready?

If it does not work

Symptom	Fix
`docker : The term 'docker' is not recognized`	install / start Docker Desktop; `docker desktop start`
`Unable to locate package` while building	the repository was cloned with Windows line endings; use the LF clone made by `build-essentials-docker`
A stage hangs	Docker Desktop has too little memory (Settings -> Resources); the stage is stopped after 20 min